Hash Table and Heap

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Oct 5, 2026

Introduction

This technical note summarizes the tools and logic used in Hash Table and heap.

Table of contents

- What Is a Good Hash Table?
- Type of Hash Tables
    - Chained Hashing
    - Open Address Hashing
- Example: Time Complexity Analysis
- Table Doubling
- What Is a valid heap?
- Heap operation
    - Insert in heap
    - Replace in heap
- Analysizing Alogrithms Using Heap
- Bonus: Leetcode Two Sum

What Is a Good Hash Table?

According to the note, Good hash functions should have the following properties:

All hash table locations are equally likely to be accessed

We should try our best to distribute keys uniformly across all buckets.

Bad Hash Table:

$$h(K) = (9K) \bmod 78$$

Since gcd(9, 78) = 3, the hash function can only produce multiples of 3 (0, 3, 6, ..., 75). Therefore, only 1/3 of the buckets can be used.

Keys with a “regular” pattern should not be mapped to the same locations

Bad Hash Table:

$$h(K) = K \bmod 10$$

We can easily see that keys 10, 20, and 30 will all be mapped to bucket 0, resulting in multiple collisions.

Hash Table Size and Modulus

In addition, the size of the hash table should be greater than or equal to the modulus. Otherwise, some hash values may exceed the valid index range. Typically, we choose the modulus to be equal to the table size.

Table Size n	Modulus m	Result
10	10	Valid
10	7	Valid, but buckets 7–9 are unused
10	13	Invalid: indices 10–12 are possible

Type of Hash Tables

There are two common ways to handle collisions in hash tables: **chained hashing** and **open addressing hashing**.

Both are valid approaches, but they use different methods to handle collisions and may have different time complexities for dictionary operations.

Chained Hashing

Chained hashing is a common way to implement a hash table. When a collision occurs, we can store multiple elements in the same bucket, typically using a linked list.

For example, suppose we have a hash table of size 5 with the following hash function:

$$h(K) = K \bmod 5$$

If we insert keys 12, 7, 22, 9, and 14, we can see that keys 12, 7, and 22 are all mapped to bucket 2, while keys 9 and 14 are mapped to bucket 4.

Instead of searching for another empty bucket, chained hashing stores the elements in a chain associated with each bucket.

The following table summarizes the five main operations in chained hashing.

Method	Description
`Insert(K, D)`	Appends a new key-data pair `(K, D)` to the linked list at `H[h(K)]`.
`Member(K)`	Returns `true` if key `K` exists in the hash table; otherwise, returns `false`.
`Retrieve(K)`	Returns the data associated with key `K`.
`Add(K, x)`	Finds key `K` and adds `x` to its current data (`e.data = e.data + x`).
`Replace(K, D)`	Finds key `K` and replaces its current data with `D`.

The time complexities of these operations are as follows:

Method	Best Case	Expected Case	Worst Case
`Insert(K, D)`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Method	Best Case	Expected Case	Worst Case
`Member(K)`	$\Theta(1)$	$\Theta(1 + s/m)$	$\Theta(s)$
`Retrieve(K)`	$\Theta(1)$	$\Theta(1 + s/m)$	$\Theta(s)$
`Add(K, x)`	$\Theta(1)$	$\Theta(1 + s/m)$	$\Theta(s)$
`Replace(K, D)`	$\Theta(1)$	$\Theta(1 + s/m)$	$\Theta(s)$

Where:

- s = Number of elements stored in the hash table.
- m = Number of buckets in the hash table.

The expected time complexity assumes uniform hashing.

Open Address Hashing

Open address hashing is another common way to handle collisions in a hash table.

Unlike chained hashing, open address hashing stores all elements directly in the hash table. When a collision occurs, we search for another available bucket using a **probing sequence**.

For example, suppose we have a hash table of size 7 with the following hash function:

$$h(K, j) = (K + j) \bmod 7$$

where $j = 0, 1, 2, \ldots$ is the number of probing attempts.

If we insert keys 10, 17, 24, 8, and 15, collisions will occur because several keys initially map to the same bucket.

Using linear probing, the final hash table will look like this:

```
Hash Table (size = 7)

Index    Key
  0       -
  1       8
  2      15
  3      10
  4      17
  5      24
  6       -
```

For example, when inserting key 17, bucket 3 is already occupied by key 10. Therefore, we increase j to 1 and try bucket 4.

Similarly, key 24 first tries buckets 3 and 4 before being stored in bucket 5.

Unlike chained hashing, no linked lists are needed because all keys are stored directly in the array.

The following table summarizes the main operations in open address hashing.

Method	Description
`Insert(K, D)`	Searches for an empty bucket using a probing sequence and stores `(K, D)` in that bucket.
`Member(K)`	Checks whether key `K` exists by following the probing sequence. Returns `true` or `false`.
`Retrieve(K)`	Searches for key `K` using the probing sequence and returns its associated data.
`Delete(K)`	Searches for key `K` and marks its bucket as deleted using `flagDeleted = true`.
`Retrieve(K)` (with deletion)	Returns the data only if key `K` is found and its deletion flag is `false`.

The time complexities of these operations are as follows:

Method	Best Case	Expected Case	Worst Case
`Insert(K, D)`	$\Theta(1)$	$\Theta(1)$	$\Theta(s)$
`Member(K)`	$\Theta(1)$	$\Theta(1)$	$\Theta(s)$
`Retrieve(K)`	$\Theta(1)$	$\Theta(1)$	$\Theta(s)$
`Delete(K)`	$\Theta(1)$	$\Theta(1)$	$\Theta(s)$
`Retrieve(K)` (with deletion)	$\Theta(1)$	$\Theta(1)$	$\Theta(s)$

Where:

- s = Number of elements stored in the hash table.
- m = Number of buckets in the hash table.
- $\alpha = s/m$ = Load factor.

Example: Time Complexity Analysis

Consider the following algorithm:

```
function Func(A[], n)
    HashTable.Init()
    x = 0

    for i = 1 to n do
        if HashTable.Member(A[i]) then
            HashTable.Add(A[i], 1)
        else
            HashTable.Insert(A[i], 10)
        end if
```

```
    end for

    x = 0

    for i = 1 to n do
        if HashTable.Member(A[i]) then
            x = x + 1
        end if
    end for

    return x
end function
```

Assume that the hash table uses **Open Address Hashing (OAH)** and that its size is at least $2n$.

We want to determine the **best, expected, and worst case running times**.

Best Case:

Each hash table operation takes $\Theta(1)$ time in the best case.

Since both loops execute n times, the total running time is:

$$T_{\text{best}}(n) = n\Theta(1) + n\Theta(1) = \boxed{\Theta(n)}$$

Expected Case:

$$ET(n) = n\Theta(1) + n\Theta(1) = \boxed{\Theta(n)}$$

Worst Case:

In the worst case, `Member` and `Add` may each take $\Theta(s)$ time due to collisions and repeated probing.

In the first loop, the number of elements increases as elements are added, while the second loop performs Member on a Hashtable containing up to n elements.

Thus, using the classroom worst-case bounds:

$$\begin{aligned} T_{\text{worst}}(n) &= \sum_{i=1}^{n} (\Theta(i) + \Theta(i)) + n\Theta(n) \\ &= \Theta(n^2) + \Theta(n^2) \\ &= \boxed{\Theta(n^2)} \end{aligned}$$

Summary:

Case	Time Complexity
Best Case	$\Theta(n)$
Expected Case	$\Theta(n)$
Worst Case	$\Theta(n^2)$

Table Doubling

When a hash table becomes full, we need to find a way to expand its capacity so that more elements can be inserted.

There are different ways to expand a hash table, such as:

- **Multiplicative expansion:** Multiply the current table size by a constant factor k (e.g., doubling the size when $k = 2$).
- **Additive expansion:** Increase the current table size by a fixed constant c.

To analyze the total running time $T(n)$, we need to consider two parts:

1. **Insertion Cost:** The time required to insert n elements into the hash table.
2. **Expansion Cost:** The time required to expand the table and rehash existing elements.

Therefore,

$$T(n) = T_{\text{insertion}}(n) + T_{\text{expansion}}(n)$$

In most problems, the time complexities of a single insertion and a table expansion are given.

Therefore, we can analyze these two costs separately and combine them to determine the total running time $T(n)$.

Let's consider the following example to see how this works.

Consider table expansion where we double the size of the hash table whenever it becomes full.

Each insertion takes $\Theta(\log s)$ time, where s is the number of elements currently in the table.

Table expansion takes ck^2 time, where k is the size of the new table.

We want to determine the **total running time of n insertions** and the **amortized cost per insertion**.

Step 1: Set Up the Running Time

As mentioned before, we need to consider two parts:

1. **Insertion Cost:** Each insertion takes $\Theta(\log i)$ time.
2. **Expansion Cost:** The table size doubles whenever it becomes full, and each expansion takes ck^2 time.

The table expands as follows:

$$1 \to 2 \to 4 \to 8 \to \cdots \to 2^k$$

To determine the number of expansions, we first let:

$$2^k = n$$

Taking the logarithm on both sides:

$$k = \log_2 n$$

For sufficiently large n, the total running time is:

$$T(n) = \sum_{i=1}^{n} \Theta(\log i) + c(2^2 + 4^2 + 8^2 + \cdots + (2^k)^2)$$

Therefore,

$$T(n) = \sum_{i=1}^{n} \Theta(\log i) + c\sum_{j=1}^{k} 4^j$$

Step 2: Upper Bound

We first write the total running time as the sum of the insertion cost and the expansion cost.

$$T(n) = \sum_{i=1}^{n} \log i + c(2^2 + 4^2 + 8^2 + \cdots + (2^k)^2)$$

Since $2^2 = 4$, $4^2 = 4^2$, and $8^2 = 4^3$, we can rewrite it as:

$$T(n) = \sum_{i=1}^{n} \log i + c(4 + 4^2 + 4^3 + \cdots + 4^k)$$

Now, we factor out the largest term 4^k:

$$T(n) = \sum_{i=1}^{n} \log i + c4^k \left(\frac{1}{4^{k-1}} + \frac{1}{4^{k-2}} + \cdots + \frac{1}{4} + 1 \right)$$

Reversing the order of the terms:

$$T(n) = \sum_{i=1}^{n} \log i + c4^k \left(1 + \frac{1}{4} + \frac{1}{4^2} + \cdots + \frac{1}{4^{k-1}} \right)$$

For the insertion cost, since $\log i \leq \log n$:

$$T(n) \leq \sum_{i=1}^{n} \log n + c4^k \left(1 + \frac{1}{4} + \frac{1}{4^2} + \cdots + \frac{1}{4^{k-1}} \right)$$

Therefore:

$$T(n) \leq n \log n + c4^k \left(1 + \frac{1}{4} + \frac{1}{4^2} + \cdots + \frac{1}{4^{k-1}} \right)$$

We can extend the finite geometric series to an infinite geometric series to obtain an upper bound:

$$T(n) \leq n \log n + c4^k \left(1 + \frac{1}{4} + \frac{1}{4^2} + \cdots \right)$$

Using the geometric series formula:

$$1 + r + r^2 + \cdots = \frac{1}{1 - r}$$

where $r = \frac{1}{4}$:

$$T(n) \leq n \log n + c4^k \cdot \frac{1}{1 - \frac{1}{4}}$$

$$T(n) \leq n \log n + c4^k \cdot \frac{4}{3}$$

Recall that:

$$2^k = n$$

Thus:

$$4^k = (2^k)^2 = n^2$$

Substituting into the inequality:

$$T(n) \leq n \log n + \frac{4c}{3} n^2$$

Since $n \log n \in O(n^2)$:

$$\boxed{T(n) \in O(n^2)}$$

Step 3: Lower Bound

We start with the total running time:

$$T(n) = \sum_{i=1}^{n} \log i + c(4 + 4^2 + 4^3 + \cdots + 4^k)$$

For the insertion cost, we only consider the last half of the terms.

Since all terms are nonnegative, removing the first half gives us a lower bound:

$$T(n) \geq \sum_{i=n/2}^{n} \log i + c4^k$$

For every $i \geq n/2$, we know that:

$$\log i \geq \log \left(\frac{n}{2} \right)$$

Therefore:

$$T(n) \geq \sum_{i=n/2}^{n} \log \left(\frac{n}{2} \right) + c4^k$$

There are at least $n/2$ terms in this summation, so:

$$T(n) \geq \frac{n}{2} \log \left(\frac{n}{2} \right) + c4^k$$

Recall that:

$$2^k = n$$

Therefore:

$$4^k = (2^k)^2 = n^2$$

Substituting this into the inequality:

$$T(n) \geq \frac{n}{2} \log \left(\frac{n}{2} \right) + cn^2$$

Since both terms are nonnegative for sufficiently large n:

$$T(n) \geq cn^2$$

Therefore:

$$\boxed{T(n) \in \Omega(n^2)}$$

Step 4: Combine the Bounds

From the upper bound:

$$T(n) \in O(n^2)$$

From the lower bound:

$$T(n) \in \Omega(n^2)$$

Therefore:

$$\boxed{T(n) \in \Theta(n^2)}$$

Step 5: Amortized Analysis

The amortized cost is the total running time divided by the number of insertions.

$$T_{\text{amortized}}(n) = \frac{T(n)}{n}$$

Since $T(n) = \Theta(n^2)$:

$$T_{\text{amortized}}(n) = \frac{\Theta(n^2)}{n}$$

Therefore:

$$\boxed{T_{\text{amortized}}(n) = \Theta(n)}$$

Summary:

Analysis	Time Complexity
Total insertion cost	$\Theta(n \log n)$
Total expansion cost	$\Theta(n^2)$
Total running time	$\Theta(n^2)$

Analysis	Time Complexity
Amortized cost per insertion	$\Theta(n)$

Although the table size doubles only occasionally, the quadratic expansion cost dominates the total running time. Therefore, the amortized cost of each insertion is linear rather than constant.

What Is a Valid Heap?

In this section, we only consider **Max-Heaps**.

A valid Max-Heap must satisfy two properties:

1. **Heap Property:** Every parent node must be greater than or equal to its children.
2. **Complete Binary Tree:** All levels are completely filled except possibly the last level, which must be filled from left to right.

For example, the following is a valid Max-Heap:

```
        90
       /  \
     70    80
    / \    /
  30 50  60
```

Each parent is greater than or equal to its children, and the tree is complete.

A heap can also be represented using an array.

Using **1-based indexing**, index 0 is not used, and the root is stored at index 1.

For a node at index i:

- Parent: $\lfloor i/2 \rfloor$
- Left child: $2i$
- Right child: $2i + 1$

For example, if the root is at index $i = 1$, its left child is at index 2 and its right child is at index 3.

The number of levels in a complete binary tree with n nodes is:

$$L = \lceil \log_2(n + 1) \rceil$$

Therefore, the height of the tree is:

$$h = L - 1 = \lfloor \log_2 n \rfloor$$

Since the height grows logarithmically with n, heap operations that move along one path from the root to a leaf (or vice versa) take at most $O(\log n)$ time.

Heap Operation

Insert in Heap

When inserting a new element into a Max-Heap, we first place it at the next available position to maintain the complete binary tree property.

If the new element is greater than its parent, we swap them.

We repeat this process until the heap property is satisfied or the element reaches the root.

This process is called **Bubble Up** (or Sift Up).

For example, suppose we insert 85 into the following Max-Heap:

Before insertion:

```
      90
     /  \
   70    80
  / \    /
 30 50  60
```

Step 1: Insert 85 at the next available position.

```
      90
     /  \
   70    80
  / \   /  \
 30 50 60  85
```

Step 2: Compare 85 with its parent 80 and swap them.

```
      90
     /  \
   70    85
  / \   /  \
 30 50 60  80
```

Since 85 is smaller than 90, no further swaps are needed.

In the worst case, the inserted element moves from the last level to the root.

Therefore:

Therefore, the running time is:

$$\boxed{T_{\text{Insert}}(s) = \Theta(\log s)}$$

ExtractMax in a Heap

In a Max-Heap, the maximum element is always stored at the root.

To extract the maximum element, we:

1. Remove the root and save its value.
2. Move the last element to the root to maintain the complete binary tree property.
3. Compare the new root with its children.
4. If the root is smaller than either child, swap it with the **larger child**.
5. Repeat until the heap property is restored.

This process is called **Bubble Down** (or Sift Down).

For example, suppose we extract the maximum element from the following heap:

Before extraction:

```
      90
     /  \
   70    85
  / \   /  \
30 50 60  80
```

Step 1: Remove 90 and move the last element (80) to the root.

```
      80
     /  \
   70    85
  / \    /
30 50  60
```

Step 2: Compare 80 with its children (70 and 85).

Since 85 is the larger child and is greater than 80, we swap them.

```
      85
     /  \
   70    80
  / \    /
30 50  60
```

The heap property is now satisfied.

In the worst case, the element at the root moves all the way down to the last level.

Therefore, the running time is:

$$\boxed{T_{\text{ExtractMax}}(s) = \Theta(\log s)}$$

Analysizing Alogrithms Using Heap

When analyzing algorithms that use a Max-Heap, we need to pay attention to the current number of elements in the heap.

Both `Insert` and `ExtractMax` have a worst-case running time of $\Theta(\log s)$, where s is the current heap size.

Unlike our previous hashing examples, we cannot simply use $\Theta(\log n)$ for every heap operation.

Instead, we should first determine **how the heap size s changes throughout the algorithm**.

Then, we can set up the summations and use the Bounding Method to determine the total running time.

Consider the following algorithm:

```
function Func(A[], n)
    P.Init()

    for i = 1 to n^2 do
        k = Random(n)

        for j = 1 to k do
            P.Insert(A[i] * A[j])
        end for
    end for

    for i = 1 to n log(n) do
        x = P.ExtractMax()
        Print x
    end for
end function
```

Assume that P is a Max-Heap and s represents the current number of elements in the priority queue.

We want to determine the **worst-case running time**.

Step 1: Determine the Range of s

Initially, the heap is empty:

$$s = 0$$

In the worst case, `Random(n)` returns n every time.

Therefore, the first nested loop performs:

$$n^2 \cdot n = n^3$$

insertions.

After all insertions:

$$s = n^3$$

Next, the second loop performs $n \log n$ extractions.

Therefore, the heap size decreases from n^3 to approximately:

$$s = n^3 - n \log n$$

We can now set up the running time based on how s changes.

Step 2: Set Up the Running Time

The first part performs n^3 insertions, with the heap size increasing from 0 to n^3.

The second part performs $n \log n$ extractions, with the heap size decreasing from n^3 to $n^3 - n \log n$.

Using the worst-case cost of $\Theta(\log s)$ for both operations, we obtain:

$$T(n) = \sum_{s=1}^{n^3} \Theta(\log s) + \sum_{s=n^3-n\log n+1}^{n^3} \Theta(\log s)$$

Now, we use the **Bounding Method** to find the upper and lower bounds.

Step 3: Upper Bound

We start with:

$$T(n) = \sum_{s=1}^{n^3} \Theta(\log s) + \sum_{s=n^3-n\log n+1}^{n^3} \Theta(\log s)$$

Since every $s \leq n^3$:

$$\log s \leq \log(n^3)$$

Therefore, using positive constant factors for the operation costs:

$$T(n) \leq C_1 \sum_{s=1}^{n^3} \log(n^3) + C_2 \sum_{s=n^3-n\log n+1}^{n^3} \log(n^3)$$

The first summation contains n^3 terms, and the second contains $n \log n$ terms.

Thus:

$$T(n) \leq C_1 n^3 \log(n^3) + C_2 n \log n \cdot \log(n^3)$$

Using:

$$\log(n^3) = 3 \log n$$

We obtain:

$$T(n) \leq 3C_1 n^3 \log n + 3C_2 n (\log n)^2$$

Since $n(\log n)^2$ grows more slowly than $n^3 \log n$:

$$\boxed{T(n) \in O(n^3 \log n)}$$

Step 4: Lower Bound

We start with:

$$T(n) = \sum_{s=1}^{n^3} \Theta(\log s) + \sum_{s=n^3 - n\log n + 1}^{n^3} \Theta(\log s)$$

Since all terms are nonnegative, we can remove some terms to obtain a lower bound.

For the insertion cost, we consider only the last half of the terms.

For the extraction cost, we also consider only the last half of the terms.

Therefore:

$$T(n) \geq c_1 \sum_{s=n^3/2}^{n^3} \log s + c_2 \sum_{s=n^3 - \frac{1}{2}n\log n + 1}^{n^3} \log s$$

For every $s \geq n^3/2$ in the first summation:

$$\log s \geq \log\left(\frac{n^3}{2}\right)$$

For every $s \geq n^3 - \frac{1}{2}n\log n + 1$ in the second summation:

$$\log s \geq \log\left(n^3 - \frac{1}{2}n\log n + 1\right)$$

Therefore:

$$T(n) \geq c_1 \frac{n^3}{2} \log\left(\frac{n^3}{2}\right) + c_2 \frac{n\log n}{2} \log\left(n^3 - \frac{1}{2}n\log n + 1\right)$$

Using the logarithm property:

$$\log\left(\frac{n^3}{2}\right) = 3\log n - \log 2$$

We obtain:

$$T(n) \geq c_1 \frac{n^3}{2}(3\log n - \log 2) + c_2 \frac{n\log n}{2} \log\left(n^3 - \frac{1}{2}n\log n + 1\right)$$

The first term grows as $\Omega(n^3 \log n)$, while the second term is nonnegative.

Therefore:

$$\boxed{T(n) \in \Omega(n^3 \log n)}$$

Step 5: Combine the Bounds

From the upper bound:

$$T(n) \in O(n^3 \log n)$$

From the lower bound:

$$T(n) \in \Omega(n^3 \log n)$$

Therefore, under the worst-case operation-cost model:

$$\boxed{T(n) \in \Theta(n^3 \log n)}$$

Key Takeaways

1. **Track the Heap Size:** Determine how the heap size s changes throughout the algorithm.
2. **Set Up the Summations:** Write the total running time based on the range of s and the number of operations.
3. **Apply the Bounding Method:**
    - **Upper Bound:** Find an upper bound for each summation.
    - **Lower Bound:** Find a lower bound by considering a subset of the terms.
4. **Combine the Bounds:** Use O and Ω to determine the final Θ bound.

Bonus: Leetcode Two Sum

Since we learned Two Sum with hash tables, why don't we take a look at the actual LeetCode **Two Sum** problem?

Question:

You are given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Intuition

It's clear that for each element `A[i]`, we need to find its pair, `target - A[i]`.

The easiest way to solve this is to use two `for` loops to check all possible pairs in the array. However, the time complexity of this approach is $\Theta(n^2)$.

Therefore, we can use a hash table to improve the efficiency.

Approach

Since a well-implemented hash table supports dictionary operations such as `member`, `retrieve`, `add`, and `delete` in expected $\Theta(1)$ time, we can use a hash table to store the numbers we have already seen.

In Java, we use `containsKey()`, `get()`, and `put()` to perform these dictionary operations.

For each `nums[i]`, we first calculate:

$$need = target - nums[i]$$

Then, we check whether `need` is already in the hash table.

- If it is, we use `get()` to retrieve its index and return the two indices.
- Otherwise, we use `put()` to store `nums[i]` as the key and `i` as its value.

Time complexity: $\Theta(n)$

Code

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];

            if (map.containsKey(need)) {
                return new int[]{map.get(need), i};
            }

            map.put(nums[i], i);
        }

        return new int[]{};
    }
}
```

Reference List

1. "*CSE2331_Midterm_3_Review*" Created by Professor Painter.
2. "*CSE2331_Heaps_Homework*" Created by Professor Painter.
3. "*CSE2331_Hashing_Homework*" Created by Professor Painter.
4. "*CSE2331_Hashing_Outline*" Created by Professor Painter.
5. "*CSE2331_Heaps_Outline*" Created by Professor Painter.
6. LeetCode. "1. Two Sum." https://leetcode.com/problems/two-sum/